

SCUOLA
NORMALE
SUPERIORE



DÁNIEL GALACZ

LOUIS URBIN

SECOND PROJECT

CLASSE DI SCIENZE
SCUOLA NORMALE SUPERIORE
PISA, JUNE 2025

Prof. Fosca Giannotti
Prof. Roberto Pellungrini

Contents

Introduction	3
1 Data Exploration and Preprocessing	5
2 Classification	10
2.1 Variational Recurrent Neural Networks (VRNN)	10
2.2 Long Short-Term Memory (LSTM)	11
2.3 Gated Recurrent Unit (GRU)	11
2.4 Best Performing Model	12
3 Forecasting	13
3.1 Stacked LSTM Architectures	13
3.2 Deep Encoder-Decoder RNN	15
Conclusion	17

Introduction

In this project, we work with the **Beijing PM_{2.5}** dataset, which contains air quality and meteorological measurements collected from multiple monitoring stations across Beijing between 2010 and 2014. The dataset includes variables such as temperature, atmospheric pressure, dew point, wind speed and direction, and most importantly, PM_{2.5} concentration levels (particulate matter with diameter less than 2.5 micrometers).

Our objective is to apply machine learning techniques to perform two main tasks:

- **Classification:** Categorize air quality levels (Good, Moderate, Unhealthy, Hazardous) based on meteorological measurements .
- **Forecasting:** Predict future PM_{2.5} concentrations based on historical data (e.g., one hour ahead).

For the air quality classification task, we adopt the thresholds defined by the *U.S. Environmental Protection Agency (EPA)*. While the official EPA Air Quality Index (AQI) includes six categories, we opt for a simplified version consisting of four levels:

- **Good:** $0-9 \mu\text{g}/\text{m}^3$
- **Moderate:** $9-35 \mu\text{g}/\text{m}^3$
- **Unhealthy:** $35-225 \mu\text{g}/\text{m}^3$
- **Hazardous:** $>225 \mu\text{g}/\text{m}^3$

This dual approach allows us to explore both the temporal dynamics of air pollution in Beijing and the predictive modeling of pollution levels, which can support environmental monitoring and public health decision-making.

EPA category	$\mu\text{g}/\text{m}^3$ of PM2.5	Our category
<i>Good</i>	0.0 to 9.0	<i>Good</i>
<i>Moderate</i>	9.1 to 35.4	<i>Moderate</i>
<i>Unhealthy for sensitive groups</i>	35.5 to 55.4	<i>Unhealthy</i>
<i>Unhealthy</i>	55.5 to 125.4	
<i>Very unhealthy</i>	125.5 to 225.4	
<i>Hazardous</i>	225.5+	<i>Hazardous</i>

Figure 1: Air quality thresholds used by EPA and our model

1

Data Exploration and Preprocessing

The dataset consists of 43,824 hourly observations recorded from 2010 to 2014 in Beijing. The variables included are:

- **No:** Row index (chronological order of observations).
- **year, month, day, hour:** Timestamps of each observation.
- **pm2.5:** PM_{2.5} concentration in $\mu\text{g}/\text{m}^3$ (target variable for prediction and classification). **This feature contains missing values.** These are imputed using the median.
- **DEWP:** Dew point temperature (in °C).
- **TEMP:** Ambient temperature (in °C).
- **PRES:** Atmospheric pressure (in hPa).
- **cbwd:** Combined wind direction (categorical; e.g., NW, NE, SE, SW, cv). 'cv' denotes calm wind or no significant direction.
- **Iws:** Cumulated wind speed (m/s).
- **Is:** Cumulated hours of snow.
- **Ir:** Cumulated hours of rain.

We also engineered an **Air Quality** variable, based on PM_{2.5} thresholds as defined in the introduction, to categorize pollution levels into discrete classes.

To begin the exploratory data analysis, we plot the distribution of key features grouped by air quality levels:

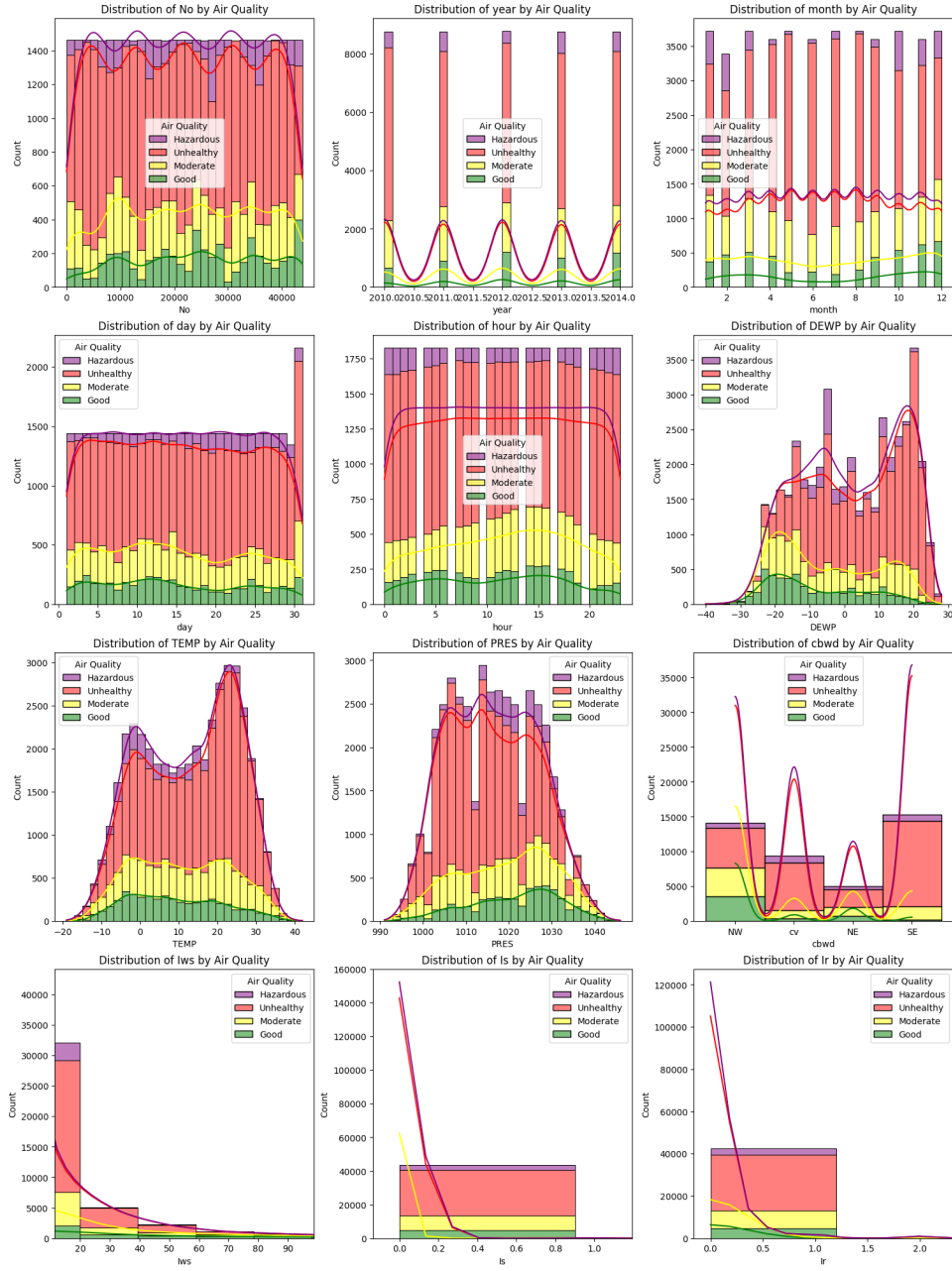


Figure 1.1: Feature distributions across air quality categories

Some remarks on the distribution plots:

- From the dew point graph, we observe that lower dew point values—indicating lower air humidity—correspond to better air quality. Conversely, when the dew point exceeds -10°C , there is a sharp decline in air quality.
- From the temperature and air pressure graphs, we can conclude that both high and low extremes are associated with high pollution levels, whereas moderate values correspond to lower pollution. Additionally, air pressure and temperature are strongly correlated, with pressure increasing as temperature decreases.

- North-West winds tend to be beneficial due to the absence of major pollution sources in that part of the city. In contrast, South-East winds are associated with poor air quality, as the southern area is heavily industrialized and consumes large amounts of coal and fossil fuels.

Next, we compute and visualize the correlation matrix for numerical features:

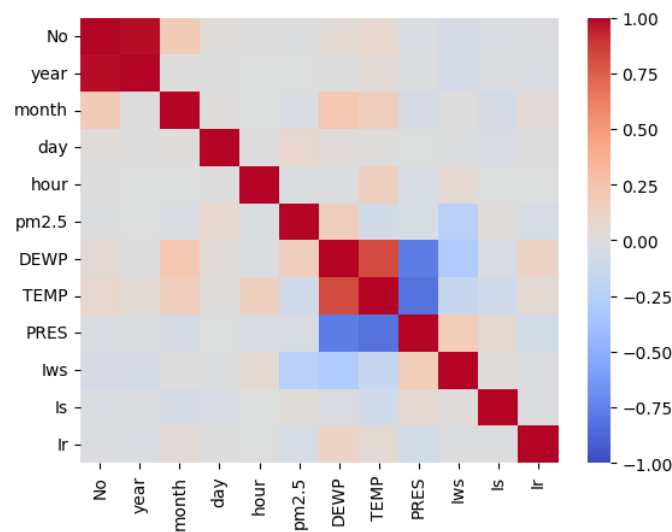


Figure 1.2: Correlation matrix between numerical variables

Observations:

- The correlation between No and year is trivial, as both represent temporal order.
- A strong positive correlation is observed between TEMP, DEWP, and PRES, which is expected due to underlying physical laws.

Given the relatively low dimensionality of the feature space, we decided **not to apply dimensionality reduction techniques** like PCA.

To further investigate temporal patterns, we plotted time series of key features:

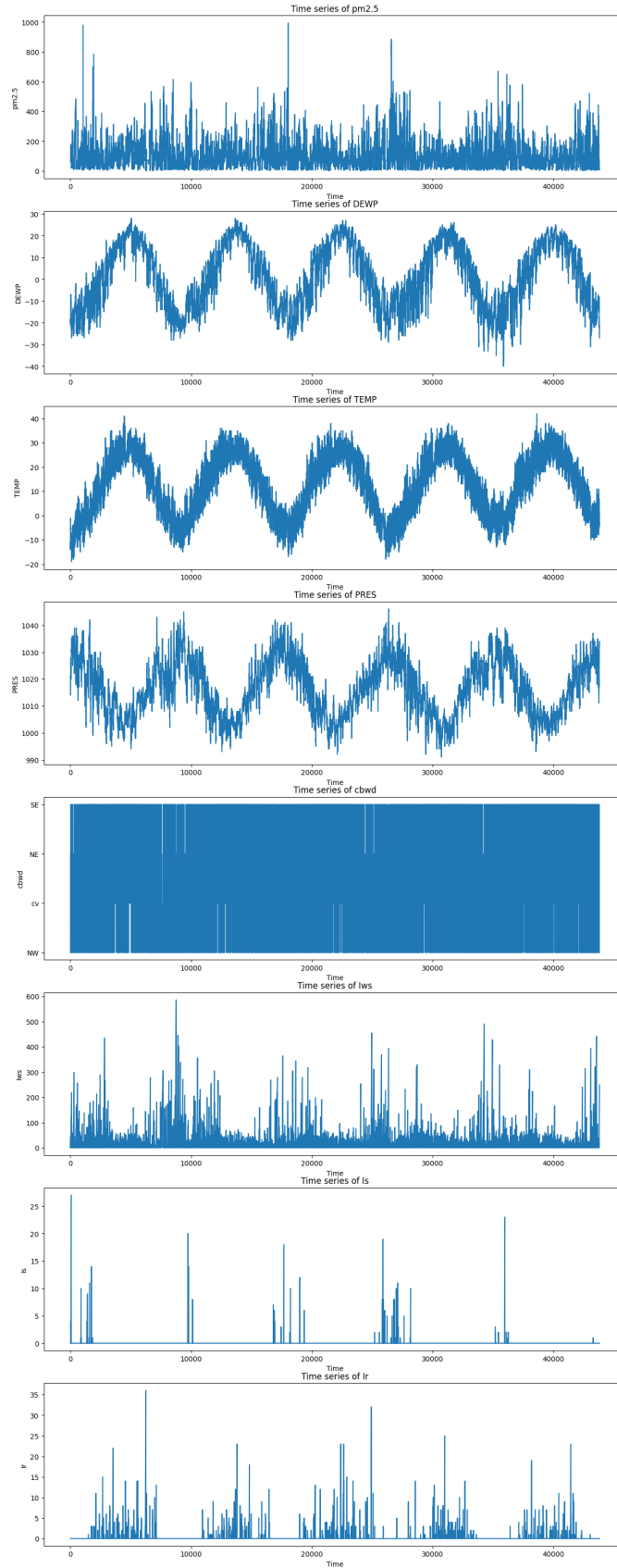


Figure 1.3: Time series of selected features over the 5-year period

We observe seasonal patterns in several variables:

- TEMP, DEWP, and PRES clearly reflect yearly cycles.
- PM_{2.5} concentrations tend to be higher every winter, partly due to residential heating, which causes a seasonal pollution peak from November to March.
- Snow hours (IS) peak during winter seasons.
- Rain hours (IR) are more prominent between may and september, corresponding to Beijing's monsoon-influenced climate.

2

Classification

2.1 Variational Recurrent Neural Networks (VRNN)

We initially experimented with a Variational Recurrent Neural Network (VRNN) to model the temporal dependencies in the data. Several hyperparameter combinations were tested, including variations in the number of layers and the dimensionality of latent variables. However, training proved unstable. In many configurations, the training loss exhibited high variance and oscillated between improvement and degradation, suggesting unstable convergence or poor optimization.

<code>hidden_size = 32</code> <code>seq_len = 128</code> <code>num_layer = 1</code> <code>num_epochs = 100</code> <code>lr = 0.0001</code>	<code>hidden_size = 32</code> <code>seq_len = 128</code> <code>num_layer = 3</code> <code>num_epochs = 100</code> <code>lr = 0.0001</code>	<code>hidden_size = 64</code> <code>seq_len = 128</code> <code>num_layer = 3</code> <code>num_epochs = 100</code> <code>lr = 0.0001</code>
Epoch [1/100], Loss: 0.8093	Epoch [1/100], Loss: 0.9096	Epoch [1/100], Loss: 1.1842
Epoch [2/100], Loss: 0.6916	Epoch [2/100], Loss: 0.6068	Epoch [2/100], Loss: 0.5502
Epoch [3/100], Loss: 0.9849	Epoch [3/100], Loss: 0.6469	Epoch [3/100], Loss: 0.9103
Epoch [4/100], Loss: 1.3306	Epoch [4/100], Loss: 0.6817	Epoch [4/100], Loss: 0.5687
Epoch [5/100], Loss: 0.7633	Epoch [5/100], Loss: 0.8285	Epoch [5/100], Loss: 0.5853
Epoch [6/100], Loss: 0.8683	Epoch [6/100], Loss: 1.0050	Epoch [6/100], Loss: 0.8937
Epoch [7/100], Loss: 1.0883	Epoch [7/100], Loss: 0.9707	Epoch [7/100], Loss: 0.9359
Epoch [8/100], Loss: 0.7740	Epoch [8/100], Loss: 1.0404	Epoch [8/100], Loss: 0.6970
Epoch [9/100], Loss: 0.8853	Epoch [9/100], Loss: 0.7793	Epoch [9/100], Loss: 0.7668
Epoch [10/100], Loss: 1.0886	Epoch [10/100], Loss: 0.8081	Epoch [10/100], Loss: 0.7186
Epoch [11/100], Loss: 0.9787	Epoch [11/100], Loss: 0.9077	Epoch [11/100], Loss: 0.9446
Epoch [12/100], Loss: 1.2311	Epoch [12/100], Loss: 0.9865	Epoch [12/100], Loss: 0.7397
Epoch [13/100], Loss: 0.9296	Epoch [13/100], Loss: 0.5645	Epoch [13/100], Loss: 0.9434
Epoch [14/100], Loss: 0.8805	Epoch [14/100], Loss: 1.0879	Epoch [14/100], Loss: 0.3489
Epoch [15/100], Loss: 0.9123	Epoch [15/100], Loss: 0.6729	Epoch [15/100], Loss: 0.5544
Epoch [16/100], Loss: 0.4626	Epoch [16/100], Loss: 0.5848	Epoch [16/100], Loss: 0.8270
Epoch [17/100], Loss: 0.7581	Epoch [17/100], Loss: 1.1640	Epoch [17/100], Loss: 0.7264
Epoch [18/100], Loss: 0.9248	Epoch [18/100], Loss: 0.7282	Epoch [18/100], Loss: 0.5745
Epoch [19/100], Loss: 0.8160	Epoch [19/100], Loss: 0.5985	Epoch [19/100], Loss: 0.9275
Epoch [20/100], Loss: 1.0200	Epoch [20/100], Loss: 0.8304	Epoch [20/100], Loss: 0.7955
Epoch [21/100], Loss: 0.7556	Epoch [21/100], Loss: 0.8710	Epoch [21/100], Loss: 0.6195
Epoch [22/100], Loss: 0.9070	Epoch [22/100], Loss: 0.5536	Epoch [22/100], Loss: 0.7572
Epoch [23/100], Loss: 0.9374	Epoch [23/100], Loss: 0.7890	Epoch [23/100], Loss: 0.7395
Epoch [24/100], Loss: 0.8815	Epoch [24/100], Loss: 0.7116	Epoch [24/100], Loss: 0.7004
Epoch [25/100], Loss: 0.9527	Epoch [25/100], Loss: 0.8877	Epoch [25/100], Loss: 1.0091
...	...	...
Epoch [97/100], Loss: 0.5856	Epoch [97/100], Loss: 0.8110	Epoch [97/100], Loss: 0.4624
Epoch [98/100], Loss: 0.9123	Epoch [98/100], Loss: 0.9252	Epoch [98/100], Loss: 0.7570
Epoch [99/100], Loss: 0.6051	Epoch [99/100], Loss: 0.5527	Epoch [99/100], Loss: 0.5247
Epoch [100/100], Loss: 0.9178	Epoch [100/100], Loss: 0.5768	Epoch [100/100], Loss: 0.6319

Figure 2.1: Training loss curves for different VRNN configurations

2.2 Long Short-Term Memory (LSTM)

We then implemented Long Short-Term Memory (LSTM) networks, which are well-suited for time-series classification tasks. Several architectures were tested with different values for:

- Number of units in the LSTM layer (e.g., 16, 32, 64),
- Sequence lengths (number of time steps fed into the model),
- Batch sizes and dropout rates.

LSTM models showed significantly better stability during training compared to VRNN. The best models reached classification accuracies of approximately 0.75 with cross-entropy loss values around 0.7, indicating a reasonable balance between fit and generalization.

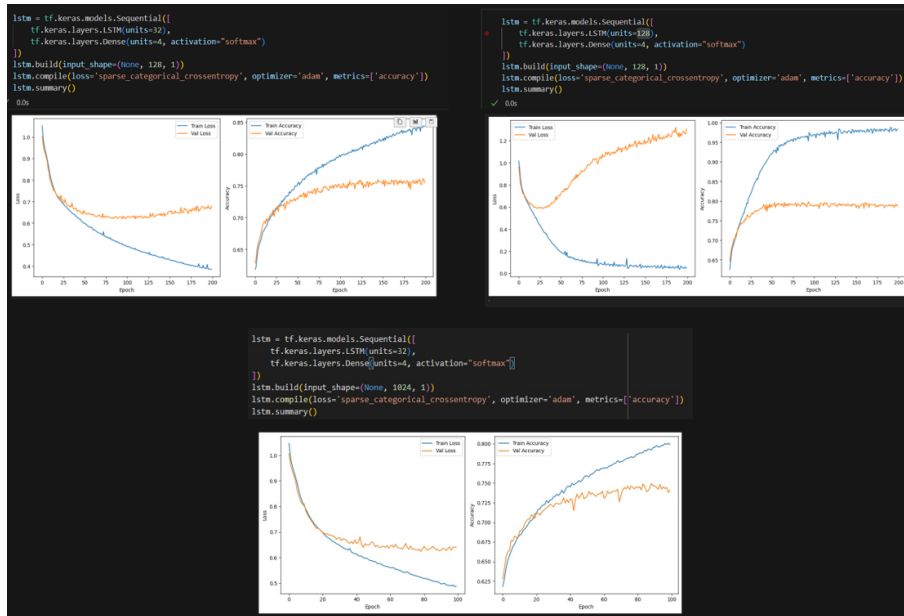


Figure 2.2: Training curves (loss and accuracy) for several LSTM configurations

2.3 Gated Recurrent Unit (GRU)

To further reduce computational complexity and potentially enhance generalization, we also evaluated Gated Recurrent Units (GRUs). GRUs are known to achieve comparable performance to LSTMs on many tasks while requiring fewer parameters.

In our experiments, the GRU-based models showed slightly faster convergence than their LSTM counterparts. Notably, our initial GRU model outperformed the LSTM, reaching an accuracy of approximately 0.78 after 100 epochs, before overfitting began to appear.

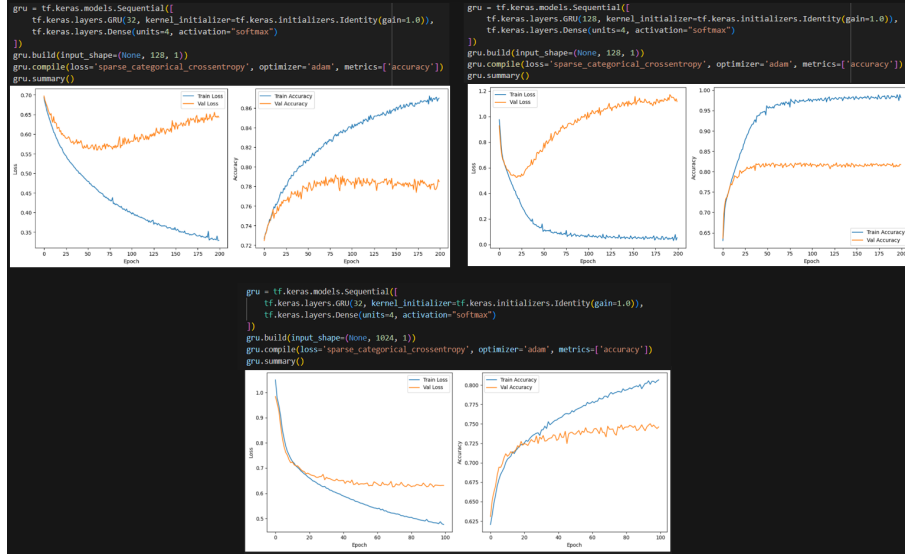


Figure 2.3: Loss and accuracy curves for the GRU-based model

2.4 Best Performing Model

Among all the tested configurations, the best-performing model was a recurrent network using either LSTM or GRU units with 32 hidden units. This setup achieved the following:

- A low and stable training loss,
- Validation accuracy of approximately 0.75 for the LSTM and 0.78 for the GRU,
- Good generalization when training was stopped at around 100 epochs.

This configuration provided the best trade-off between performance, generalization capability, and training time.

3

Forecasting

For the forecasting task, we experimented with different input window sizes and model architectures to predict future $\text{PM}_{2.5}$ values based on historical sequences.

3.1 Stacked LSTM Architectures

We began with relatively simple models: stacked LSTM networks with 1, 2, and 3 layers. After several experiments, we found that an input sequence length (**window size**) of **100 time steps** provided the best trade-off between accuracy and generalization performance.

Figure 3.1 shows the predictions from deeper LSTM models.

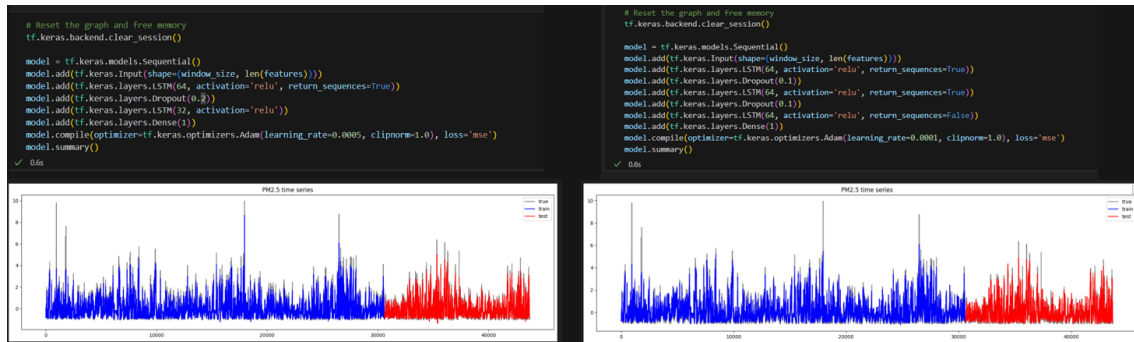


Figure 3.1: Predictions using 2-layer and 3-layer LSTM architectures

These models struggled to capture extreme $\text{PM}_{2.5}$ values — both high peaks and sharp drops — in both training and validation sets. This may be due to overfitting or a loss of information across deeper time-unfolded layers.

Surprisingly, the best results were achieved with a much simpler model: a single LSTM layer with 64 neurons.

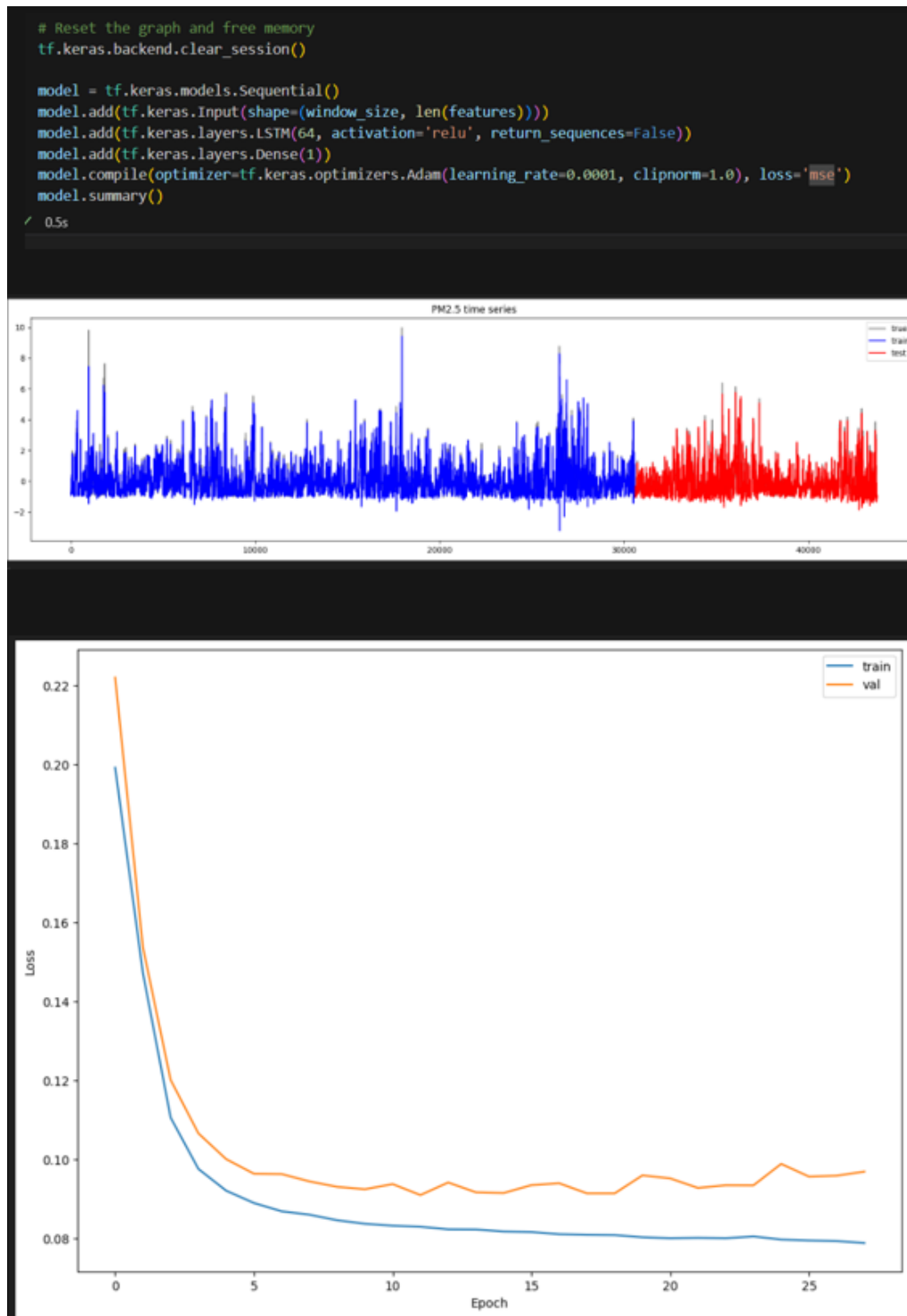


Figure 3.2: Prediction using a single-layer LSTM (64 units)

This model captured temporal dependencies more effectively, especially during high-pollution episodes. The test loss reached approximately **0.10** (mean squared error), with better alignment to true values, particularly in extreme cases. This simplicity likely helped generalization by reducing overfitting risk.

3.2 Deep Encoder-Decoder RNN

We also tested a **deep encoder-decoder RNN** for sequence-to-sequence forecasting. This architecture uses one recurrent network (the encoder) to process the input sequence and another (the decoder) to generate the output.

To prevent overfitting, we applied L2 regularization and used the `glorot_uniform` initializer, which is known to be effective for sigmoid activations in recurrent layers.

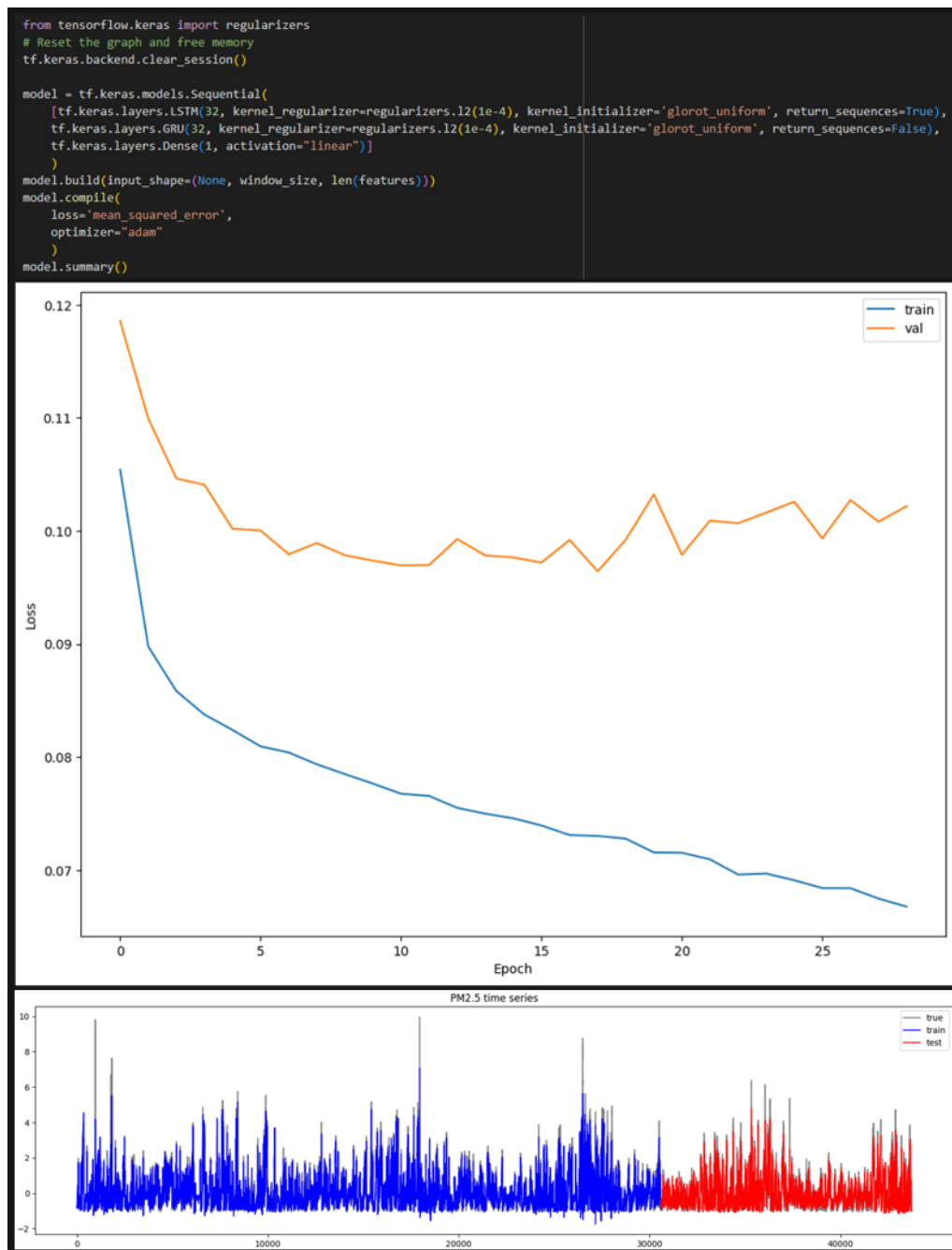


Figure 3.3: Predictions using a deep encoder-decoder RNN architecture

The encoder-decoder model achieved a test loss comparable to the single-layer LSTM (around **0.10**). Interestingly, it converged in slightly fewer epochs. However, its predictions were less accurate for extreme pollution events, suggesting that it captured general trends well but lacked precision in volatile segments of the time series.

Conclusion

In this project, we applied a range of machine learning algorithms to the Beijing PM_{2.5} dataset with the aim of classifying air quality levels and forecasting future pollution concentrations. Through experimentation with recurrent neural architectures, we gained valuable insights into both the challenges and potentials of modelling environmental time-series data. We demonstrated that Long Short-Term Memory (LSTM) networks outperformed more complex or simplified alternatives. While the Variational Recurrent Neural Network (VRNN) struggled with convergence due to high variance and temporal noise, and the Gated Recurrent Unit (GRU) suffered from overfitting, a well-tuned LSTM model achieved a classification accuracy of approximately 75 percent with good generalization. For forecasting PM_{2.5} concentrations, we found that simpler models often offered better performance. A single-layer LSTM with 64 units consistently outperformed deeper architectures, capturing short- and medium-term trends effectively while avoiding overfitting. While the encoder-decoder RNN was competitive in terms of average error, it struggled with predicting extreme pollution events—an area of particular importance for public health applications.